MedViz Project

MIRI - SCIENTIFIC VISUALIZATION

Introduction

In scientific visualization, volume rendering is widely used to represent 3D scalar fields. These scalar fields are typically uniform grids of values that may represent, for example, charge density around a molecule, medical imaging data such as MRI or CT scans, or airflow around an aircraft. Volume rendering is conceptually straightforward: by sampling the data along rays cast from the camera and assigning a color and transparency to each sample, we can generate informative and visually appealing images of these scalar fields.

In a GPU-based renderer, 3D scalar fields are commonly stored as 3D textures. Fortunately, WebGL2 (the technology we will use for this project) supports this functionality.

To begin working on the project, you must access the GitHub Classroom repository using the following link:https://classroom.github.com/a/QcEEi5UH

Each student will obtain their own copy of the project skeleton and the instructions required to run it. From there, you will implement your work primarily in the shader files.

Please remember **to commit and push your work regularly**. The frequency and consistency of your contributions will be taken into account during the evaluation. We will review your GitHub activity throughout the development of the project. Do not leave everything for the final days—this will result in a penalty.

You also have access to a **special feedback pull request** where you can ask questions and receive guidance. Feel free to use it; it is there to help you.

Submission

The deadline for the submission is **December 24th at 06:00**. This is a **hard deadline**, and no late submissions will be accepted.

Your submission must include the following items:

- **The final version of your tutorial** on how to use **SimpleITK** for data preparation. This version should incorporate the feedback you received from your peers.
- **The vertex and fragment shaders** implementing:
  - A volume rendering algorithm.
  - A soft-shadow algorithm integrated into volume rendering.
- **A 4-page report** containing:

  - INTRODUCTION: An introduction describing the context and goals of the project.
  - RELATED WORK: A brief theory section summarizing the concepts (and PW) used in this project.
  - METHOD: A detailed explanation of your implementation, including justifications for your design decisions.
  - RESULTS: The obtained results and a discussion of them.
  - ANALYSIS: An analysis, supported by experimental results, of how the transfer function and the lighting algorithm affect performance. Additionally, discuss how the two shadowing strategies influence the quality of the lighting results.
  - CONCLUSIONS: Conclusions of your work.
- **Your full code** in your GitHub Classroom repository.
- **A short video** (maximum 5 minutes) showcasing your application. The video must demonstrate:
  - How your application works.
  - The advantages and limitations you identified in the technique and/or in your implementation.
  - Anything you consider we should see of your work.

Data preparation for Volume Rendering [15% grade]

In the previous exercise, you explored the importance of preprocessing volumetric data and successfully completed that task. For the continuation of this project, it is essential that the raw files generated by your code **strictly follow the structure described below**. This consistency will ensure that your data can be correctly loaded and rendered during the volume-rendering stage.

Consider the following example file: **example_6_5_4.raw**. The dataset is organized with:

- **6 elements along the Z dimension**,
- **5 elements along the Y dimension**, and
- **4 elements along the X dimension**.

All values are written **in a single line**, without separators other than spaces.

The data is structured in **blocks of 20 consecutive values** (because 5 × 4 = 20, i.e., one full XY-slice). Within each block:

- All values share the **same integer part** (0, 1, 2, …).
- The **decimal part increases** in steps of 0.05, from .00 to .95.

Below is the example, formatted into two columns for readability (your .raw file must still contain a single line):

```
// Block 1 (values 0.00-0.95)
0.00 0.05 0.10 0.15
0.20 0.25 0.30 0.35
0.40 0.45 0.50 0.55
0.60 0.65 0.70 0.75
0.80 0.85 0.90 0.95

// Block 2 (values 1.00-1.95)
1.00 1.05 1.10 1.15
1.20 1.25 1.30 1.35
1.40 1.45 1.50 1.55
1.60 1.65 1.70 1.75
1.80 1.85 1.90 1.95

// Block 3 (values 2.00-2.95)
2.00 2.05 2.10 2.15
2.20 2.25 2.30 2.35
2.40 2.45 2.50 2.55
2.60 2.65 2.70 2.75
2.80 2.85 2.90 2.95
```

```
// Block 4 (values 3.00-3.95)
3.00 3.05 3.10 3.15
3.20 3.25 3.30 3.35
3.40 3.45 3.50 3.55
3.60 3.65 3.70 3.75
3.80 3.85 3.90 3.95

// Block 5 (values 4.00-4.95)
4.00 4.05 4.10 4.15
4.20 4.25 4.30 4.35
4.40 4.45 4.50 4.55
4.60 4.65 4.70 4.75
4.80 4.85 4.90 4.95

// Block 6 (values 5.00-5.95)
5.00 5.05 5.10 5.15
5.20 5.25 5.30 5.35
5.40 5.45 5.50 5.55
5.60 5.65 5.70 5.75
5.80 5.85 5.90 5.95
```

Make sure that **your generated raw file follows the same structure**, adapted to the dimensions and values of your dataset.

This model will serve as a starting point for testing and debugging your code. However, your final submission must be capable of correctly displaying **this volume** as well as **any other volume dataset**, after being converted to this format.

Volume Rendering [45% grade]

Theory

Generating a true-to-life image from volumetric data involves capturing the absorption, emission, and scattering of light rays within the medium (refer to Figure 1). Although modeling light transport at this level yields visually stunning and physically accurate images, it is prohibitively costly for real-time rendering—the primary goal in interactive visualization software. In scientific visualization, the ultimate aim is to empower scientists to explore their data interactively, facilitating the formulation and answering of research questions.

Because a full physically based model incorporating scattering is impractical for interactive rendering, visualization applications often adopt a simplified **emission–absorption model**. In this approach, the computationally expensive scattering effects are either neglected or approximated efficiently. In this project, we focus specifically on the emission–absorption model.

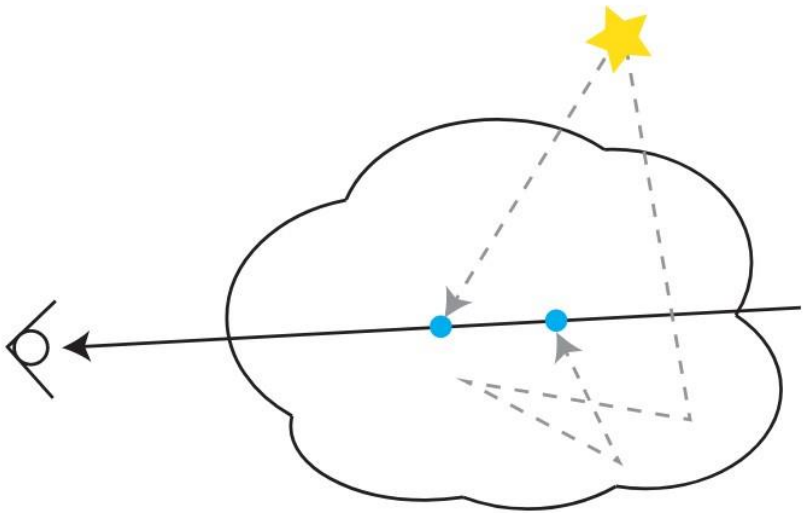

Figure 1: Physically based volume rendering, accounting for absorption and light emission by the volume, along with scattering effects.

In the emission–absorption model, we compute lighting effects only along the **primary ray** (black ray in Figure 1), ignoring contributions from secondary rays (dashed gray rays). Rays traversing the volume and reaching the eye gather the color emitted by the volume while being attenuated due to absorption.

By tracing rays from the eye through the volume, we calculate the light received by the eye by integrating emission and absorption along the ray. If a ray enters the volume at at s = 0, and exits at s = L, the light reaching the eye is given by the integral:

$$C(r) = \int_0^L C(s)\mu(s)e^{-\int_0^s \mu(t)dt}ds$$

where:

- $C(s)$ is the emitted color at point *s* along the ray,
- $\mu(s)$ is the absorption coefficient of the medium at point *s*,
- $e^{\left\{-\int_0^s \mu(t)dt\right\}}$ represents the attenuation of light from point *s* back to the eye.

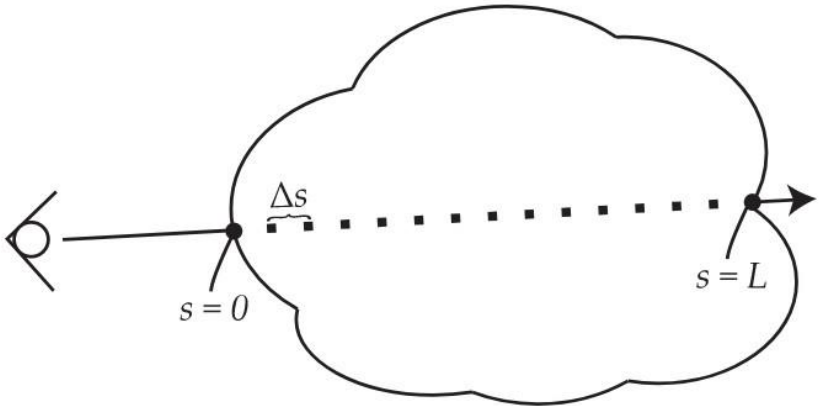

Figure 2: Computing the emission-absorption rendering integral on a volume.

As a general rule, this integral cannot be computed analytically, so we approximate it numerically. We sample the ray at **N discrete points** spaced Δs apart ($s = 0, \Delta s, 2\Delta s, \ldots, L$). See Figure 2. The continuous integral is replaced by a sum over these samples. At each sample, the attenuation term becomes a product series, progressively accumulating absorption from previous samples.

$$C(r) = \sum_{i=0}^{N} C(i\Delta s)\mu(i\Delta s)\Delta s \prod_{j=0}^{i-1} e^{-\mu(j\Delta s)\Delta s}$$

For further simplification, we approximate the attenuation term using a **Taylor series** and define $\alpha(i\Delta s) = \mu(i\Delta s)\Delta s$. This leads to the **front-to-back alpha compositing equation**.

$$C(r) = \sum_{i=0}^{N} C(i\Delta s)\alpha(i\Delta s) \prod_{j=0}^{i-1} (1 - \alpha(j\Delta s))$$

In practice, this equation is implemented as a loop along the ray:

1. Initialize accumulated color `col = 0` and accumulated opacity `acc = 0`.
2. For each sample i:
    a. Compute the premultiplied color:
    $$\hat{C}(i\Delta s) = C(i\Delta s) \cdot \alpha(i\Delta s)$$
    b. Update the accumulated color and opacity
    $$\hat{C}_i = \hat{C}_{i-1} + (1 - \alpha_{i-1})\hat{C}(i\Delta s)$$
    $$\alpha_i = \alpha_{i-1} + (1 - \alpha_{i-1})\alpha(i\Delta s)$$
3. Stop the loop if the ray exits the volume or if opacity reaches 1.

This iterative front-to-back compositing ensures correct blending and allows for **early ray termination**, improving performance.

To render an image, we trace a ray from the eye through each pixel and execute this loop for every ray intersecting the volume. Because each ray is processed independently, the method is highly parallelizable. Implementing the raymarching process in a **fragment shader** leverages the GPU's parallel computing capabilities, enabling an efficient and interactive volume renderer.

What you need to implement

You are required to implement a volume rendering algorithm using the provided skeleton, modifying **only the shaders**. Your implementation must include:

1. **User-defined transfer function**
    a. Map scalar values to color and opacity.
    b. Correctly apply the transfer function at every sample along the ray.
2. **Entry and exit point computation for each ray**
    a. Compute the points where rays intersect the volume to limit sampling to the volume region.
3. **Sampling rate**
    a. Choose a sampling step Δs according to the Nyquist–Shannon theorem to avoid aliasing and sampling distortions.
    b. Document and justify your choice.
4. **Early ray termination**
    a. Stop marching rays when accumulated opacity reaches >= `0.95`.
5. **Correct compositing**
    a. Use **premultiplied alpha** for blending: $\hat{C}(i\Delta s) = C(i\Delta s) \cdot \alpha(i\Delta s)$
    b. Ensure numerical stability and consistency across different transfer functions and datasets.

Lighting in volume rendering [40% grade]

Theory

Shadows are crucial for achieving realistic visualizations. Traditionally, shadows are generated using rasterization and **shadow mapping**, which stores depth in a texture. While computationally inexpensive, this method has limitations in memory consumption and depth precision, leading to artifacts. Furthermore, implementing shadow mapping in **volume rendering** is challenging because the medium itself contributes to light attenuation.

An alternative is **ray tracing**, which simulates the physical behavior of light to generate realistic shadows. The main challenge is maintaining **real-time performance**. Typically, this is addressed by tracing a limited number of rays and applying denoising. In this project, we focus on **approximated ray-traced shadows**, but without any denoising step.

Tracing rays toward each light source allows us to compute **pixel-accurate hard shadows**. However, in reality, hard shadows rarely occur, because real light sources have area. This gives rise to **umbra** (fully shadowed region) and **penumbra** (partial shadow region), producing soft shadows.

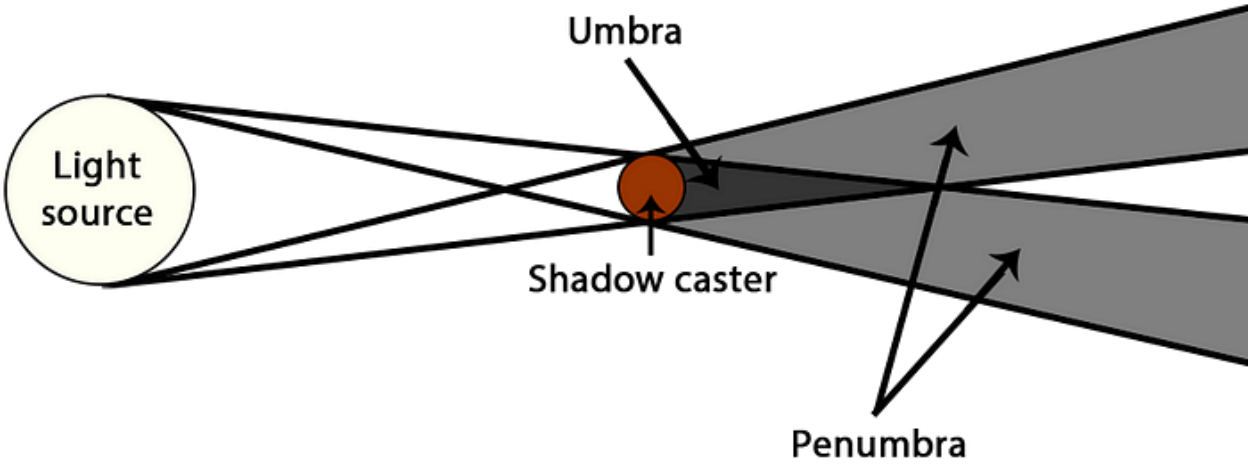

Figure 3: Schema of a light and the produced shadows.

Hard shadows occur only with point lights or lights at an infinite distance. The sun, approximated as an infinitely distant light, can generate near-perfect hard shadows. For other light sources, **soft shadows** must be simulated. Accurate soft shadows require integrating over all points on the light source, but tracing infinitely many rays is impractical. Therefore, we approximate by tracing a **limited number of rays per pixel (SPP)** and evaluating the impact on performance and visual quality.

Each shadow ray is cast toward a randomly selected point on a **spherical light source**, uniformly sampled over its surface (or disc). This selection uses linear algebra and trigonometry to define the ray direction. Occlusion in volume rendering is computed by accumulating the **opacity along the ray path**.

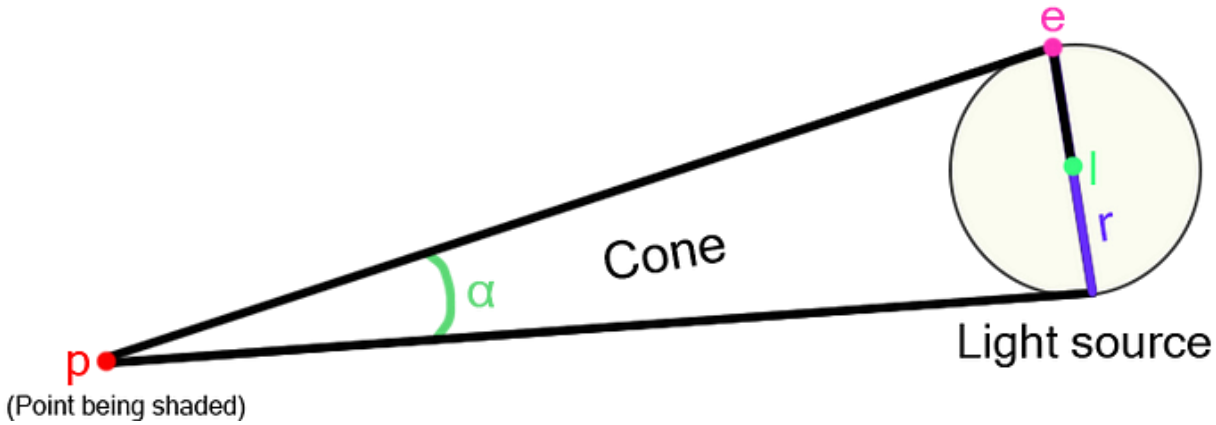

Figure 4: Sampling cone in which all the traced rays lie.

In addition to ray-based occlusion, **voxel normals** are needed for lighting calculations. The normal vector represents the direction perpendicular to the surface at each voxel and is computed from the **gradient of the scalar field**, now denoted as I:

$$\mathbf{N} = \frac{\mathbf{I}}{\|\mathbf{I}\|} = \frac{\left(\frac{\partial S}{\partial x}, \frac{\partial S}{\partial y}, \frac{\partial S}{\partial z}\right)}{\sqrt{\left(\frac{\partial S}{\partial x}\right)^2 + \left(\frac{\partial S}{\partial y}\right)^2 + \left(\frac{\partial S}{\partial z}\right)^2}}$$

Here:

- I is the gradient of the scalar field S.
- N is the normalized gradient (voxel normal)
- $|I|$ is the magnitude of the gradient vector.

This can be computed as:

$$grad(I) = \nabla I = \frac{(right - left)}{2x}, \frac{(top - bottom)}{2x}, \frac{(front - back)}{2x}$$

What you need to implement

You are required to implement a **lighting algorithm** for volume rendering using the skeleton provided, with modifications only allowed in the shaders. The requirements are:

1. **Visibility computation**
   - For each sample with opacity above a threshold (you must discuss the effect of this threshold on performance and quality), compute the **visibility VVV** of the light.
   - Trace NNN rays to a single circular or spherical light source to estimate visibility.
   - The light position is specified in **spherical coordinates** (Λ,Φ\Lambda, \PhiΛ,Φ) relative to the volume center, with a radius defining the area light.
2. **Behavior based on the number of rays traced**:
   - **0 rays:** Lighting algorithm disabled.
   - **1 ray:** Trace a single ray to the center of the light → hard shadows only.
   - **>1 rays:** Trace multiple uniformly distributed random rays to the light → soft shadows (umbra + penumbra).
3. **Occlusion strategies**:
   - Option 1: Percentage of rays that reach full opacity along their path.
   - Option 2: Normalized accumulation of reached opacities across all rays.
   - Implement both and compare in the report.
4. **Shading model**
   - Assume the volume is **Lambertian**.
   - For each sample, attenuate the color based on the light direction L and the voxel normal N:

$$\hat{C}(i\Delta s) = C(i\Delta s) \cdot V(L) \cdot (N \cdot L)$$

Here:

- $\hat{C}(i\Delta s)$ → premultiplied color after shading
- $C(i\Delta s)$ → original color from transfer function
- V(L) → light visibility (from ray tracing)
- N·L → Lambertian cosine term
- V(L) must have a minimum value to prevent fully black shadows.

5. **Normals**
   - Compute voxel normals as the **normalized gradient** of the scalar field.
   - Use central differences or similar methods to compute it.